

MODULE 04 · DOMAIN 2 - GENERATING AND READING SQL

Inherited Code

Eight hundred lines of stored procedure, no documentation, and the author left last year.

DOMAIN 2 of 5 MODULE 4 of 11 PROMPTS 3

What you are actually facing

- ◆ **It works.** It has run every night for years. That is evidence, not documentation.
- ◆ **It is layered.** Temp tables, unions, a sequence of updates after the insert.
- ◆ **Every oddity looks deliberate.** You cannot tell a workaround from a design choice.
- ◆ **You have to change it anyway.** Usually by Friday.

THE REAL RISK

The risk is not that you fail to understand it. The risk is that you understand it wrongly and confidently.

Two ways in

THE DEFAULT

Read it yourself

- Start at line one
- Lose the thread around line 200
- Build a partial mental model
- Change the part you understood

WHAT WE BUILD

Map, then narrate

- Get the shape before the detail
- Narrate one target at a time
- Find what the code cannot tell you
- Then decide what to change

THE TIEBREAKER

Reading it yourself is not wrong. It is just the slowest possible order to do things in.

Map the structure before you ask what anything **means**.

Why shape comes before meaning

- ◆ A long procedure is a sequence of steps, not a query. Each step produces something.
- ◆ Temp tables, unions and post-insert updates hide the real data flow.
- ◆ The map is short enough to check by eye, so you can trust what comes next.
- ◆ Ask what it means first and you get a confident summary you have no way to verify.

THE POINT

You are buying something checkable before you start relying on something that is not.

What you actually type

Deliberately shallow. Do not let it explain anything yet.

```
Map the structure of the stored procedure below. Do not
explain business meaning or summarise what it is for.
```

```
List every step in execution order. For each step give:
```

- What it does, what it writes to, what it reads from

```
Then list every UNION branch separately, and for each branch
state which source table drives it and which filter selects it.
```

```
Then list any column whose value differs between branches.
```

```
Numbered list. No prose. Under 400 words.
```

THE BLOCK THAT EARNS ITS PLACE

The third block finds polymorphic columns, and that is where the bugs live.

One column, three meanings

Work order dimension. Three union branches, abbreviated.

```
01 -- Branch 1 WHERE wotyp IN ('C1','C2','C3')    corrective
02     NVL(UPPER(wauf.assetid),'') AS wo_ref_id
03 -- Branch 2 WHERE wotyp IN ('P1','P2')          preventive
04     NVL(UPPER(wauf.assetid),'') AS wo_ref_id
05 -- Branch 3 WHERE wotyp IN ('J1','J2')          project
06     NVL(UPPER(wauf.contrid),'') AS wo_ref_id
07
08 -- Downstream, the bridge joins on it with no type filter:
09 INNER JOIN dim_asset_fs ast ON ast.asset_number = wo.wo_ref_id
10 -- A contract id that matches an asset number creates rows.
```

WHAT THE MAP BOUGHT YOU

Two branches put an asset in that column. The third puts a contract. Nothing downstream knows.

Where do the keys come from?

The structure map surfaced all three of these together.

```
01 max_corrective_key  bigint := 0;
02 max_preventive_key  bigint := 40000000;
03 max_project_key     bigint := 70000000;
04 -- SELECT NVL(MAX(work_order_key), ...) INTO ... <- commented out
05
06 TRUNCATE TABLE dim_work_order_fs;
07 ... max_corrective_key + ROW_NUMBER() OVER (order by 1)
08
09 -- order by 1 is a constant. It does not determine an order.
10 -- So every reload assigns different keys to the same rows.
```

THE CONDITION

Safe only if nothing outside this procedure ever stored one of these keys.

Why that matters, and when it does not

- ◆ **Fine if the keys never leave.** Rebuilt each night, joined within the same load.
- ◆ **Broken if anything stored one.** A saved report, an extract, a materialised view.
- ◆ **Also a collision risk.** Exceed forty million corrective orders and you overrun the next band.
- ◆ **One query settles it.** Grep for the key column outside this schema.

THE CHECK

You cannot tell from the code whether this is a defect. You can find out in ten minutes.

Narrate the transformations. One target table at a time.

The trap in narration

- ◆ **Restating the SQL is not narration.** 'Coalesces the field to empty string' tells you nothing.
- ◆ You want the effect on the data, not a reading of the syntax.
- ◆ Ask what values arrive, and what values leave.
- ◆ Ask which rows are dropped, and by which predicate.

THE TEST

If the narration could be produced without seeing the data model, it is a translation, not an explanation.

What you actually type

Scope to one target table. Repeat for each.

For target table <name> only, explain what the procedure does to the data. Work column by column, in INSERT list order.

For each column state:

- Which source column or columns feed it
- What happens to the value on the way through
- What it holds when the source is null or missing

Then separately, list every predicate that removes rows, and say what kind of row each one removes.

Do not restate the SQL in English. If a line's only effect is syntactic, say 'no data effect' and move on.

THE HALF PEOPLE SKIP

The row-removal list finds filters that quietly changed scope years ago.

What narration cannot tell you

- ◆ Whether an inner join was chosen deliberately or pasted from elsewhere.
- ◆ Whether a hard-coded code list is current.
- ◆ Why a filter excludes one storage location and not another.
- ◆ It can tell you all three exist. That is the useful part.

THE BOUNDARY

Same boundary as module two. Behaviour is recoverable from code. Intent is not.

Dead code is a drift signal

Three things in this procedure that never execute

- ◆ **Commented incremental filters.** A timestamp range that no longer applies.
- ◆ **A commented soft-delete block.** So the deleted flag is always the same value.
- ◆ **Cleanup after the exception handler.** The handler re-raises, so it never runs.
- ◆ **Each one marks a decision** somebody made and did not finish or record.

WHAT IT COSTS

Downstream filters on that deleted flag are no-ops. Nobody knows, because they still read correctly.

Blast radius. And an honest warning about it.

The caveat first

- ◆ **A model cannot see your warehouse.** It knows only what you paste in.
- ◆ Asked what depends on a table, it will produce a plausible list from names.
- ◆ That list is a hypothesis. It is not lineage and it never was.
- ◆ Feed it real evidence and the same prompt becomes genuinely useful.

READ THIS FIRST

This is the prompt in the library most likely to give you a confident answer with nothing behind it.

What you actually type

Note the first line. It changes what comes back.

```
I am pasting evidence below, not asking you to recall anything.  
Use only what is in this message.
```

1. The procedure that loads <table>
2. Grep results for <table> across our repository
3. The list of views and reports we know read this schema

```
Given only that, list what would be affected if I changed  
the grain of <table>. For each item state:  
- Which piece of evidence tells you it depends on this table  
- What specifically would break, not just that it would
```

```
Then list what you would need to see to be more confident.
```

```
Do not include anything you cannot point to in the evidence.
```

THE RULE THAT FIXES IT

Every item must cite its evidence. That one rule removes the guessing.

If you have no lineage tooling

Most teams do not. This still works

- ◆ **Grep the repository** for the table name. Crude and surprisingly complete.
- ◆ **Query the system tables** for view definitions referencing it.
- ◆ **Check query history** for who has selected from it in ninety days.
- ◆ Paste all three in. Ten minutes of gathering beats any amount of prompting.

IN PRACTICE

The prompt is only as good as the evidence. Gathering the evidence is the actual work.

The structure map reports that the procedure truncates its target, inserts, and then runs three separate update statements against the same table to populate dimension keys. You have been asked to add one column. You have not read the procedure line by line yet.

What does the structure map tell you to check first?

- ☐ A Whether the three updates could be folded into the insert
- ☐ B Whether the truncate and insert are in a single transaction
- ☐ C Whether your new column needs to be in the insert or an update
- ☐ D Whether the updates are slowing the nightly run

◆ PAUSE AND DECIDE

What does the structure map tell you to check first?

- A

 Fold the updates in
A refactor you were not asked for, on a procedure you do not understand yet.
- B

 Transaction boundary
Truncate commits implicitly on most warehouses, so a mid-load failure leaves the table empty.
- C

 Where your column goes
You do need to answer this. It is a smaller question than whether the load is safe at all.
- D

 Are the updates slow
Probably yes, and it is not what the map just told you. Performance is a later module.

THE SIGNAL

The map surfaced a shape, and that shape has a known failure mode. Notice it before you add to it.

The transformation narration says a column is populated from a source field, upper cased, with nulls replaced by empty string. It describes this for every column in turn.

It reads well. It is roughly nine hundred words. You are no clearer on what the table is for.

What went wrong?

- ☐ A The procedure is genuinely just simple mapping
- ☐ B You got a translation of the syntax rather than an explanation
- ☐ C The prompt should have asked for business context
- ☐ D The target table was too large to scope in one prompt

◆ PAUSE AND DECIDE

What went wrong?

- A

 It really is simple
Possible, and you can test it in seconds. Ask for the row removal list and see if it is empty.
- B

 Translation, not explanation
Nine hundred words that could have been written without the data model. That is the tell.
- C

 Ask for business context
It would invent it. The code does not contain business context and neither does the model.
- D

 Scope was too large
Scope was right. The output was long because it padded, not because the table is complex.

THE SIGNAL

The row-removal half of that prompt is what carries the meaning. A missing filter list is the symptom.

You paste the procedure and ask what depends on the table. You get eight items back, including two views and a monthly finance report, all named plausibly.

You pasted no grep results and no consumer list. Your change ships Friday.

How much of that list can you rely on?

- ☐ A All of it. The names match our conventions closely
- ☐ B The views, since view names follow the table name pattern
- ☐ C None of it. Treat every item as a lead to verify
- ☐ D Only items it expressed high confidence about

◆ PAUSE AND DECIDE

How much of that list can you rely on?

- A

 All of it
The names match conventions because it generated them from your conventions. That is circular.
- B

 The views at least
The most seductive wrong answer. A plausible view name is still a generated view name.
- C

 None of it, verify each
Eight leads is genuinely useful. Eight facts would be dangerous. Know which you have.
- D

 The high confidence ones
Stated confidence is not calibrated against evidence it never had. It measures nothing here.

THE SIGNAL

The list is not worthless. It is a checklist to verify, and it took ten seconds to produce.

The structure map finds a column fed by different source fields in different union branches. You confirm a downstream bridge joins it with no branch filter.

The procedure has run nightly for four years. Nobody has ever reported a problem with the bridge.

What do you conclude?

- ☐ A It is fine. Four years without incident is strong evidence
- ☐ B It is a live bug. Fix the bridge before your change ships
- ☐ C It is a latent bug. Test for collisions, then decide
- ☐ D It is deliberate. Assume the original author knew something

◆ PAUSE AND DECIDE

What do you conclude?

- A

 Four years is evidence
Evidence that no collision has occurred yet. Not that the identifier spaces cannot overlap.
- B

 Live bug, fix it now
You do not know it is live. Fixing an unmeasured problem inside an unrelated change is its own risk.
- C

 Latent. Test, then decide
One count query settles it, and the answer changes what you do next entirely.
- D

 Assume it was deliberate
This is how every defect in this course survived. Absent evidence, deference is just a guess.

THE SIGNAL

One query tells you whether the identifier spaces overlap today. Run it before you argue about it.

Take this to work tomorrow

- ◆ Next unfamiliar procedure, run the structure map before you read a line of it.
- ◆ Narrate one target table. Check the row removal list came back populated.
- ◆ Before any blast radius question, spend ten minutes gathering grep and query history.
- ◆ Anything the code cannot explain goes in a comment, even if you never fix it.

THE PAYOFF

That last habit is what stops the next person inheriting the same mystery you did.

You can recover **what** the code does. You cannot recover **why**.